CP312 ASSIGNMENT 3

I certify that this Assignment is entirely my own work and that all external resources, if used, have been properly acknowledged

1.

Recurrence Relation:

C = Array representing the values of the coins

C[i] = Value of the ith coin

OPT[i,w] = Max. Value 1st person can obtain using subset of first i coins, such that no 2 chosen coins are adjacent, and total value doesn't exceed a weight limit of W.

↳ Base cases:

↳ OPT(0,w) = 0 for all $w \geq 0$ (0 if there are no coins)

↳ OPT(i,w) = 0 if $w < 0$ (invalid state, capacity constraint was exceeded)

↳ Recursive step:

For the ith coin, the first person has 2 choices:

↳ Skip the ith coin: The maximum value is simply the optimal solution for the first i-1 coins with the same capacity w.
OPT(i-1, w)

↳ pick the ith coin: The person gains the value C[i]. Because they picked coin i, they are forbidden from picking coin i-1. Therefore they must look at the optimal solution for the first i-2 coins, and their remaining capacity becomes w - C[i].
This is only valid if $w \geq C[i]$. → C[i] + OPT(i-2, w-C[i])

The recurrence relation is:

$$OPT(i,w) = \max \begin{cases} OPT(i-1,w) \\ C[i] + OPT(i-2, w-C[i]) & \text{if } w \geq C[i] \end{cases}$$

Pseudocode:

```
FUNCTION MaxCoinValue(i, w):
    IF i<=0 OR w==0:
        RETURN 0                    // Base cases
    IF memo[i][w] != -1:            //check if value is computed
        RETURN memo[i][w]
    skip_coin = MaxCoinValue(i-1, w) // option 1; skip coin
    take_coin = 0
    IF w >= c[i]:                   // option 2; take coin (if capacity allows)
        take_coin = c[i] + MaxCoinValue(i-1, w-c[i])
    memo[i][w] = MAx(skip_coin, take_coin)
    RETURN memo[i][w]
```

DP trace table:

Assume we have 3 coins with values $C=[3,2,4]$ and our capacity constraint is $K=5$.

i \ w	0	1	2	3	4	5
0 (No coins)	0	0	0	0	0	0
1 ($c_1=3$)	0	0	0	3	3	3
2 ($c_2=2$)	0	0	2	3	3	3
3 ($c_3=4$)	0	0	2	3	4	4

Time complexity:

↳ State Space: The Memoization table has dimensions $n \times K$, representing the number of coins n and the maximum capacity K. There are $O(nK)$ unique states.

↳ Time per State: Calculating the maximum between the 2 choices takes $O(1)$ constant time.

↳ Total Time complexity: The overall time complexity of this memoized dynamic algorithm is $O(nK)$

2.

a)

Recurrence Relation:

DP(w) = The Min. number of packages needed to obtain exactly w units.

$S\{s_1, s_2, \ldots, s_n\}$ = The set of available package sizes

↳ Base case: DP(0) = 0 (0 packages are needed to get 0 units)

↳ Recursive step: $DP(w) = 1 + \min_{s_i \in S, s_i \leq w} \{DP(w - s_i)\}$

↳ If no combination can form exactly w, DP(w) = ∞

Dp Trace Table:

w	Process / Min. calculation	DP(w)
0	Base case	0
1	No Package ≤ 1	∞
2	1 + DP(2-2) → 1 + DP(0)	1
3	1 + DP(3-3) → 1 + DP(0)	1
4	1 + DP(4-2) → 1 + DP(2)	2
5	1 + min(DP(5-2), DP(5-3)) → 1 + min(DP(3), DP(2))	2
6	1 + min(DP(6-2), DP(6-3)) → 1 + min(DP(4), DP(3))	2

b)

Python file is attached in submission as P2.py